

Полиморфизъм и динамично свързване в C++

Семинар 13
Серхан

Статично свързване - функция се избира по време на компилация

Динамично свързване - функция се избира по време на изпълнение, чрез виртуална таблица

Виртуална функция - декларирана с `virtual` в базовия клас; наследниците могат да я предефинират

абстрактен клас - има поне една чисто виртуална функция (`= 0`); не се създават обекти директно от него

полиморфизъм - едно и също нещо (действие) се реализира по различен начин в зависимост от типа на обекта

Статично свързване	Динамично свързване
- compile time - по-бързо - ниска гъвкавост - при обикновени функции	- runtime - с <code>virtual</code> - по-бавно (заради таблицата) - висока гъвкавост - при полиморфни йерархии

Виртуални функции

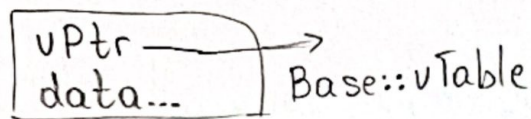
- декларира се с `virtual` в базовия клас
- наследниците предефинират с `override`, в противен случай се ползва версията на базовия клас
- викат се чрез `Base*` и `Base&` за полиморфизъм
- `= 0` - чисто виртуална
- виртуален деструктор на базовия клас

Виртуална таблица - `vTable` и `vPtr`

При клас с виртуална функция:

1. Създава се `vTable` - адреси на виртуалните функ. за всеки клас
2. Добавя скрит `vPtr` - указател към `vTable` - към всеки обект

Обект от Base:

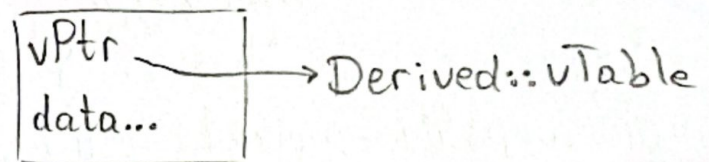


Base::vTable:

[0] → Base::f()

[1] → Base::g()

Обект от Derived:



Derived::vTable:

[0] → Derived::f() ← предефинирана

[1] → Base::g() ← наследена

Стъпки при виртуално извикване:

Base *p = new ThirdDerived();
p → f();

① p сочи към обект от Third

② f() е виртуална ⇒ vTable

③ от обекта взимаме vPtr (той сочи към vTable)

④ на позиция [p] е Second::f

⑤ извикваме Second::f(*p)

ThirdDerived::vTable:

[f] → Second::f

[g] → First::g

[h] → Third::h

Полиморфизъм - класът има поне една виртуална функция

- std::vector<std::unique_ptr<Base>> - използваме го защото при излизане от scope-а, всичко се почиства автоматично (не пишем delete)
- shared_ptr - когато един обект ще се споделя между няколко структури
- Виртуален деструктор - задължителен!
на базовия клас

Абстрактни класове - ~~сдържат само виртуални функции~~; има поне една чисто виртуална функция

Абстрактен наследник - не е предефинирал чисто виртуалната функция на базовия; не могат да се създават директно обекти

Интерфейс - само чисто виртуални функции, без данни

override - защита от грешки

- хваща грешки (например: Base няма тази функция или няма тази функция с този параметър)

final - забрана за наследяване / предефиниране

```
class A final {
```

// логика

```
}
```

```
class B {
```

```
public:
```

```
virtual void f() { cout << B::f(); }
```

```
virtual ~B() = default;
```

```
};
```

→ класът A не може да се наследява

```
class C: public B
```

```
{ void f() final { cout << C::f(); }
```

```
};
```

↓
повече тази функция не може да се предефинира от наследниците на C

Bugobe casting

- static_cast - compile time, без runtime проверка, по-бърз
- dynamic_cast - runtime проверка, при полиморфни типове, по-бавен
- const_cast - добавя/маха const
- reinterpret_cast - ниско ниво, опасен

Up-cast: $\text{Derived}^* \rightarrow \text{Base}^*$ (безопасен)

Down-cast: $\text{Base}^* \rightarrow \text{Derived}^*$ (опасен
търбва cast)

Base
down (опасно) ↓
Derived ↑ up (безопасно)

Factory Design Pattern

Фабрика

- Начин за създаване на обекти, при който не знаем предварително точния клас на обекта, а само неговия интерфейс. Фабричният метод взима решението кой обект да създаде.

- Без Factory - повтарящ се код навсякъде и трудна поддръжка

```
class UserFactory {  
public:  
    static unique_ptr <User> createUser (const string & type, const string & name)  
    {  
        if (type = "guest") return make_unique <Guest> ();  
        if (type = "admin") return make_unique <Admin> ();  
        if (type = "user") return make_unique <RegisteredUser> (name);  
        return nullptr;  
    }  
}
```

→ няма нужда от delete ръчно

Observer Design Pattern

Наблюдател

- Начин за организиране на комуникация между обекти. Един обект уведомява автоматично всички регистрирани обекти при промяна на своето състояние.

Пример:

- Вестник публикува новина
- Абонатите получават известие при всяка нова новина
- Абонатите могат да се абонират и отписват
- Използва се при:
 - промяна в един обект трябва да се отрази в други
 - не знаеш колко обекта ще реагират
 - обектите са слабо свързани

- Алгоритъм: трябва да направим 2 интерфейса - за Observer и за Subject
 - ↓ virtual функции:
да има абониране, отписване,
известие и virtual дестр.
 - ↓ виртуално,
да има функция
за update и
virtual дестр.

- Използва се shared_ptr при по-сложни системи

Observer & Factory

↓
кой да е уведомен
при промяна
1 издател и много
абонати

↓
кой обект да се създаде
1 производител и много продукти